



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 3 – Árboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 – “B”
Alumnos participantes:	Quispe Nasimba Daniela Alejandra
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con árboles e implementar diferentes operaciones sobre árboles utilizando C++ y Java para comprender su estructura, funcionamiento y aplicaciones dentro de las estructuras de datos.

Específicos:

- Comprender y aplicar la estructura y funcionamiento de los árboles N-arios y árboles binarios mediante la creación, manipulación y recorrido de nodos, utilizando conceptos fundamentales de estructuras de datos como jerarquía, relaciones padre-hijo y organización dinámica de información en los lenguajes C++ y Java.
- Implementar algoritmos recursivos para realizar operaciones sobre árboles, tales como conteo de nodos, cálculo de altura, recorridos y transformaciones estructurales, fortaleciendo la lógica de programación y el análisis de problemas relacionados con estructuras jerárquicas y recursividad.
- Comparar la implementación de árboles en C++ y Java identificando diferencias en el manejo de memoria, uso de punteros y referencias, estructuras dinámicas y administración de objetos, con el fin de comprender las ventajas y características particulares de cada lenguaje en el desarrollo de estructuras de datos.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

➤ **Estructura del repositorio**

El repositorio contiene 5 ejercicios, cada uno debe ser hecho en C++ y Java.

1. Ejercicio 1: Árboles Básicos (Conteo de nodos en árboles N-arios).
2. Ejercicio 2: Árbol Binario (Inserción en BST).
3. Ejercicio 3: Árbol Binario (Cálculo de profundidad máxima).
4. Ejercicio 4: Recorridos (Implementación de In-Order).
5. Ejercicio 5: Transformación (Inversión o árbol espejo).

➤ **Instrucciones para el desarrollo**

1. Dentro de cada archivo encontrarán la estructura básica de las clases (o structs) y la definición de un método específico que deben completar.
2. Localizar el comentario “Todo: Implementa tu lógica aquí”. Esa es la única sección del código que necesitan modificar.



3. No es necesario modificar el método “main”. Este método ya contiene la construcción de un árbol de prueba y las impresiones necesarias para validar que su algoritmo funciona correctamente.
4. Su objetivo es lograr que, al ejecutar el código, los resultados calculados coincidan con los resultados esperados impresos en la consola.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computadora portátil.
- IDE.
- JDK.
- C++.
- Visual Studio Code.
- Internet.
- GitHub.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

- Analizar la estructura base proporcionada en cada ejercicio para comprender la organización de los nodos, las relaciones entre padres e hijos y el funcionamiento general de los árboles N-arios y árboles binarios tanto en C++ como en Java.
- Implementar la lógica faltante en cada método señalado con el comentario “TODO: Implementa tu lógica aquí”, aplicando correctamente conceptos de recursividad, manejo de nodos y estructuras dinámicas de datos.
- Desarrollar la función de conteo de nodos en árboles N-arios, recorriendo todos los hijos de cada nodo hasta contabilizar completamente la estructura del árbol.
- Implementar la inserción de nodos dentro de un Árbol Binario de Búsqueda (BST), respetando las reglas de organización donde los valores menores se ubican a la izquierda y los mayores o iguales a la derecha.
- Programar el cálculo de la profundidad o altura máxima del árbol binario utilizando funciones recursivas para determinar la cantidad máxima de niveles existentes en la estructura.
- Implementar el recorrido In-Order en árboles binarios para recorrer correctamente los nodos siguiendo el orden izquierdo, raíz y derecha, permitiendo obtener los valores ordenados en el caso de un BST.
- Desarrollar la transformación del árbol binario mediante la inversión de sus nodos hijos, creando un árbol espejo a partir de la estructura original.
- Realizar pruebas de ejecución utilizando los árboles definidos en el método main, verificando que los resultados obtenidos coincidan con los valores esperados mostrados en consola.
- Comparar las diferencias de implementación entre C++ y Java, especialmente en el manejo de memoria, uso de punteros y referencias, estructuras dinámicas y administración de objetos.



- Documentar y analizar el funcionamiento de cada algoritmo implementado, identificando la lógica utilizada, el uso de recursividad y la complejidad básica de cada operación realizada sobre los árboles.

2.7 Resultados obtenidos

Link de GitHub:

https://github.com/Daniela-Quispe/APE-3_Arboles

Implementación de Arboles

1. INTRODUCCION:

En esta práctica se trabajó con estructuras de datos tipo árbol utilizando los lenguajes de programación C++ y Java. El desarrollo de los ejercicios permitió comprender la organización jerárquica de los árboles, el uso de nodos enlazados mediante referencias o punteros y la aplicación de recursividad para resolver distintos problemas.

Los árboles son una de las estructuras de datos más importantes dentro de la informática debido a que permiten representar información de manera organizada y eficiente. Son ampliamente utilizados en sistemas de archivos, bases de datos, inteligencia artificial, compiladores y motores de búsqueda.

Durante la práctica se implementaron diferentes operaciones fundamentales sobre árboles N-arios y árboles binarios, incluyendo:

- Conteo de nodos.
- Inserción en árboles binarios de búsqueda.
- Cálculo de altura.
- Recorridos.
- Transformación de árboles.

Cada ejercicio fue desarrollado tanto en C++ como en Java, permitiendo comparar el manejo de memoria, objetos, punteros y referencias entre ambos lenguajes.

2. MARCO TEORICO:

Árbol:

Un árbol es una estructura de datos no lineal formada por nodos conectados jerárquicamente. Cada árbol posee:

- Un nodo principal llamado raíz.
- Nodos hijos.
- Nodos padres.
- Subárboles.
- Nodos hoja.

La relación entre nodos permite representar información jerárquica.

Características de los árboles:

Los árboles poseen varias características importantes:

- Son estructuras dinámicas.
- Permiten organizar información jerárquicamente.
- Facilitan búsquedas eficientes.
- Utilizan nodos enlazados.
- Pueden recorrerse mediante algoritmos recursivos.

Los árboles son ampliamente utilizados en sistemas informáticos debido a su capacidad para representar relaciones jerárquicas de manera eficiente.



Aplicaciones de los árboles:

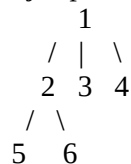
Los árboles tienen numerosas aplicaciones dentro de la informática, entre ellas:

- Sistemas de archivos.
- Motores de búsqueda.
- Bases de datos.
- Compiladores.
- Inteligencia artificial.
- Redes computacionales.
- Árboles genealógicos.
- Sistemas de decisión.

Árbol N-ario:

Un árbol N-ario es una estructura donde cada nodo puede tener múltiples hijos. No existe un límite fijo de descendientes.

Ejemplo:



En este tipo de árbol:

- El nodo 1 es la raíz.
- Los nodos 2, 3 y 4 son hijos de la raíz.
- Los nodos 5 y 6 son hijos del nodo 2.

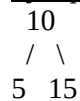
Los árboles N-arios son útiles para representar estructuras donde un elemento puede tener múltiples dependencias.

Árbol Binario:

Un árbol binario es una estructura donde cada nodo posee como máximo dos hijos:

- Hijo izquierdo.
- Hijo derecho.

Ejemplo:

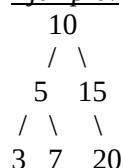


Árbol Binario de Búsqueda (BST):

El Árbol Binario de Búsqueda, conocido como BST (Binary Search Tree), es un tipo especial de árbol binario que organiza los valores siguiendo reglas específicas:

- Todos los valores menores al nodo actual se colocan a la izquierda.
- Todos los valores mayores o iguales se colocan a la derecha.

Ejemplo:



Los BST permiten realizar:

- Inserciones eficientes.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



- Búsquedas rápidas.
- Eliminación organizada de nodos.
- Recorridos ordenados.

Recursividad:

La recursividad es una técnica de programación donde una función se llama a sí misma para resolver un problema más grande dividiéndolo en subproblemas más pequeños.

La recursividad es fundamental en árboles debido a que:

- Cada subárbol puede tratarse como un árbol independiente.
- Simplifica la implementación de recorridos.
- Reduce la complejidad lógica del código.

Toda función recursiva debe poseer:

1. Caso base.
2. Llamada recursiva.

Ejemplo base:

Si el nodo es null → terminar.

Si no → procesar y llamar nuevamente.

Recorridos de Árboles Binarios:

Un recorrido es el proceso mediante el cual se visitan todos los nodos del árbol siguiendo un orden específico.

Los recorridos permiten:

- Mostrar información.
- Buscar datos.
- Procesar estructuras jerárquicas.
- Organizar información.

Existen dos tipos principales:

- DFS (Depth First Search)
- BFS (Breadth First Search)

DFS (Depth First Search):

DFS significa “búsqueda en profundidad”. Este método recorre primero las ramas más profundas del árbol antes de regresar a otros nodos.

Los recorridos DFS son:

- Inorden
- Preorden
- Postorden

DFS generalmente utiliza:

- Recursividad
- Pila implícita del sistema

Recorrido Inorden:

El recorrido Inorden sigue el orden: Izquierda → Raíz → Derecha

Recorrido Preorden:

El recorrido Preorden sigue el orden: Raíz → Izquierda → Derecha

Recorrido Postorden:

El recorrido Postorden sigue el orden: Izquierda → Derecha → Raíz



BFS (Breadth First Search):

BFS significa “búsqueda en anchura”. Este recorrido visita los nodos nivel por nivel.

Altura de un árbol:

La altura de un árbol representa la cantidad máxima de niveles existentes desde la raíz hasta el nodo más profundo.

Ejemplo:

```
1
 \
  2
   \
    3
```

La altura del árbol es 3. El cálculo de altura normalmente se realiza utilizando recursividad.

Árbol Espejo:

Un árbol espejo es el resultado de intercambiar todos los hijos izquierdos y derechos de cada nodo del árbol.

Ejemplo:

Árbol original:

```
1
 / \
2   3
```

Árbol invertido:

```
1
 / \
3   2
```

La transformación se realiza mediante intercambio recursivo de nodos.

Árboles Binarios en C++:

En C++ los árboles binarios se implementan utilizando:

- estructuras (struct)
- punteros
- memoria dinámica (new)

Ejemplo:

*Nodo** izquierdo;

*Nodo** derecho;

Árboles Binarios en Java:

En Java los árboles binarios se implementan mediante:

- clases
- objetos
- referencias

Ejemplo:

Nodo izquierdo;

Nodo derecho;

3. DESARROLLO DE LOS EJERCICIOS:



EJERCICIO 1:

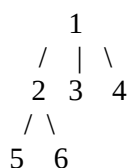
Conteo de nodos en un árbol N-ario:

Descripción del ejercicio:

En este ejercicio se desarrolló una función capaz de contar todos los nodos existentes dentro de un árbol N-ario utilizando recursividad.

El árbol N-ario se caracteriza porque cada nodo puede tener múltiples hijos, por lo tanto, fue necesario recorrer completamente cada subárbol para contabilizar todos los nodos existentes.

El árbol utilizado en las pruebas fue el siguiente:



Desarrollo:

Para resolver el problema se utilizó una función recursiva que realiza las siguientes operaciones:

1. Verifica si el nodo actual es nulo.
2. Cuenta el nodo actual.
3. Recorre cada hijo del nodo.
4. Llama nuevamente a la función para contar los nodos hijos.
5. Retorna la suma total de nodos.

La recursividad permitió recorrer automáticamente todos los niveles del árbol sin necesidad de utilizar estructuras auxiliares complejas.

Resultado obtenido:

Nodos esperados: 6

Nodos calculados: 6

EJERCICIO 2:

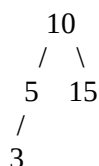
Insertión en Árbol Binario de Búsqueda (BST):

Descripción del ejercicio:

En este ejercicio se implementó la inserción de nodos dentro de un Árbol Binario de Búsqueda. El BST organiza los datos siguiendo reglas específicas:

- Valores menores → lado izquierdo.
- Valores mayores o iguales → lado derecho.

El árbol generado fue:



Desarrollo:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



La función de inserción realiza comparaciones entre el valor a insertar y el valor del nodo actual. El proceso implementado fue:

1. Verificar si el nodo es nulo.
2. Comparar el nuevo valor con el valor actual.
3. Si el valor es menor:
4. Continuar por el subárbol izquierdo.
5. Si el valor es mayor o igual:
6. Continuar por el subárbol derecho.
7. Insertar el nodo cuando se encuentre una posición vacía.

La implementación se realizó mediante recursividad para simplificar el recorrido del árbol.

Resultado obtenido:

Raíz: 10

Hijo izquierdo: 5

Hijo derecho: 15

Hijo izquierdo del 5: 3

EJERCICIO 3:

Cálculo de altura del árbol:

Descripción del ejercicio:

El objetivo de este ejercicio fue calcular la altura máxima de un árbol binario.

La altura representa el número máximo de niveles existentes desde la raíz hasta el nodo más profundo.

Árbol utilizado:

```
1
 \
  2
 /
3
```

Desarrollo:

La función implementada realiza:

1. Verificación de nodo nulo.
2. Cálculo de altura izquierda.
3. Cálculo de altura derecha.
4. Comparación de ambas alturas.
5. Retorno de la mayor altura más uno.

La lógica recursiva permitió recorrer completamente el árbol hasta encontrar el nivel más profundo.

Resultado obtenido:

Altura esperada: 3

Altura calculada: 3

EJERCICIO 4:

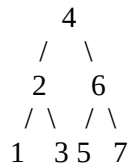
Recorrido In-Order:



Descripción del ejercicio:

En este ejercicio se implementó el recorrido In-Order sobre un árbol binario.

Árbol utilizado:



El recorrido In-Order sigue el siguiente orden:

Izquierda → Raíz → Derecha

Desarrollo:

La función recursiva desarrollada realiza:

1. Recorrer el subárbol izquierdo.
2. Guardar el valor del nodo actual.
3. Recorrer el subárbol derecho.

Cada valor visitado se almacenó dentro de una lista dinámica para mostrar el recorrido completo. En un BST este recorrido devuelve los valores ordenados de menor a mayor.

Resultado obtenido:

[1, 2, 3, 4, 5, 6, 7]

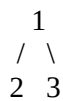
EJERCICIO 5:

Transformación de árbol espejo:

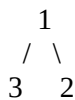
Descripción del ejercicio:

En este ejercicio se implementó la transformación de un árbol binario en su árbol espejo.

Árbol original:



Árbol invertido:



Desarrollo:

La función desarrollada realiza:

1. Intercambio del hijo izquierdo y derecho.
2. Aplicación recursiva sobre el subárbol izquierdo.
3. Aplicación recursiva sobre el subárbol derecho.

La recursividad permitió realizar la inversión en todos los niveles del árbol automáticamente.

Resultado obtenido:



Izquierdo: 3
Derecho: 2

4. IMPLEMENTACION DE LOS EJERCICIOS EN C++:

EJERCICIO 1:

Conteo de nodos en árbol N-ario:

En C++ se utilizó una estructura struct llamada NodoN para representar cada nodo del árbol. Cada nodo almacenaba:

- Un valor entero.
- Un vector dinámico de hijos mediante vector<NodoN*>.

```
struct NodoN {  
    int valor;  
    vector<NodoN*> hijos;  
};
```

Función para contar nodos:

```
int contarNodos(NodoN* raiz) {  
  
    if (raiz == nullptr) {    // Caso base  
        return 0;  
    }  
    int cantidad = 1;    // Contar nodo actual  
    for (NodoN* hijo : raiz->hijos) {    // Recorrer hijos  
        cantidad += contarNodos(hijo);  
    }  
    return cantidad;  
}
```

La función *contarNodos()* se implementó utilizando recursividad.

La lógica consistió en:

1. Verificar si el nodo era nulo.
2. Contar el nodo actual.
3. Recorrer todos los hijos utilizando un ciclo.
4. Sumar recursivamente la cantidad de nodos hijos.

EJERCICIO 2:

Inserción en Árbol Binario de Búsqueda:

En C++ se utilizó una estructura Nodo con punteros hacia el hijo izquierdo y derecho.

```
struct Nodo {  
    int valor;  
    Nodo* izquierdo;  
    Nodo* derecho;  
};
```

Función para insertar nodos:

```
Nodo* insertar(Nodo* raiz, int valor) { // Función insertar
```



```
if (raiz == nullptr) { // Si está vacío crear nodo
    return new Nodo(valor);
}
if (valor < raiz->valor) { // Insertar izquierda
    raiz->izquierdo = insertar(raiz->izquierdo, valor);
} else { // Insertar derecha
    raiz->derecho = insertar(raiz->derecho, valor);
}
return raiz;
}
```

La función insertar() realizaba comparaciones entre el valor nuevo y el nodo actual. La lógica implementada fue:

- Valores menores → izquierda.
- Valores mayores o iguales → derecha.

La función recorría el árbol recursivamente hasta encontrar una posición vacía (nullptr) donde insertar el nuevo nodo.

EJERCICIO 3:

Cálculo de altura del árbol:

En C++ se implementó una función recursiva llamada calcularAltura().

La lógica desarrollada fue:

1. Retornar 0 si el nodo es nulo.
2. Calcular la altura izquierda.
3. Calcular la altura derecha.
4. Retornar la altura mayor más uno.

Se utilizó la función max() de la librería <algorithm>.

```
return 1 + max(alturaIzquierda, alturaDerecha);
```

Función para calcular altura:

```
int calcularAltura(Nodo* raiz) { // Función altura

    if (raiz == nullptr) { // Caso base
        return 0;
    }
    int izquierda = calcularAltura(raiz->izquierdo); // Altura izquierda
    int derecha = calcularAltura(raiz->derecho); // Altura derecha
    return 1 + max(izquierda, derecha); // Retornar altura mayor
}
```

La recursividad permitió encontrar automáticamente el nivel más profundo del árbol.

EJERCICIO 4:

Recorrido In-Order:

En C++ se utilizó un vector<int> para almacenar el recorrido.



`vector<int> resultado;`

La función recursiva siguió el orden:

➤ Izquierda → Nodo → Derecha

Función para realizar el recorrido in-order:

```
void inOrderAux(Nodo* nodo, vector<int>& resultado) { // Función auxiliar

    if (nodo == nullptr) { // Caso base
        return;
    }
    inOrderAux(nodo->izquierdo, resultado); // Izquierda
    resultado.push_back(nodo->valor); // Nodo actual
    inOrderAux(nodo->derecho, resultado); // Derecha

}

vector<int> recorridoInOrder(Nodo* raiz) { // Función principal
    vector<int> resultado;
    inOrderAux(raiz, resultado);
    return resultado;
}
```

La implementación permitió recorrer correctamente todos los nodos del árbol binario. En árboles BST este recorrido devuelve los elementos ordenados.

EJERCICIO 5:

Inversión de árbol binario:

En C++ se desarrolló una función para invertir el árbol binario intercambiando los hijos izquierdos y derechos.

La lógica aplicada fue:

1. Guardar temporalmente el hijo izquierdo.
2. Intercambiar ambos hijos.
3. Aplicar recursividad sobre cada subárbol.

```
Nodo* temp = raiz->izquierdo;
raiz->izquierdo = raiz->derecho;
raiz->derecho = temp;
```

Función para invertir un árbol binario:

```
Nodo* invertir(Nodo* raiz) { // Función invertir

    if (raiz == nullptr) { // Caso base
        return nullptr;
    }
    Nodo* temporal = raiz->izquierdo; // Intercambio
    raiz->izquierdo = raiz->derecho;
    raiz->derecho = temporal;

    invertir(raiz->izquierdo); // Recursividad
    invertir(raiz->derecho);
}
```



```
    return raiz;  
}
```

La función recorrió todos los niveles del árbol realizando la inversión completa.

5. IMPLEMENTACION DE LOS EJERCICIOS EN JAVA:

EJERCICIO 1:

Conteo de nodos en árbol N-ario:

En Java se utilizó una clase *NodoN*. Cada nodo almacenaba:

- Un valor entero.
- Una lista dinámica de hijos utilizando *List<NodoN>*.

```
class NodoN {  
    int valor;  
    List<NodoN> hijos;  
}
```

Función para contar nodos:

```
public static int contarNodos(NodoN raiz) {  
  
    if (raiz == null) { // Caso base  
        return 0;  
    }  
    int cantidad = 1; // Contar nodo actual  
  
    for (NodoN hijo : raiz.hijos) { // Recorrer hijos  
        cantidad += contarNodos(hijo);  
    }  
    return cantidad;  
}
```

La función *contarNodos()* utilizó la misma lógica recursiva implementada en C++. Java permitió trabajar utilizando referencias a objetos sin necesidad de manejar memoria manualmente.

EJERCICIO 2:

Inserción en Árbol Binario de Búsqueda:

En Java se utilizó una clase *Nodo* con referencias a objetos.

```
class Nodo {  
    int valor;  
    Nodo izquierdo;  
    Nodo derecho;  
}
```

Función para insertar nodos:

```
public static Nodo insertar(
```



```
Nodo raiz,  
int valor) {  
  
    if (raiz == null) { // Crear nodo si está vacío  
        return new Nodo(valor);  
    }  
    if (valor < raiz.valor) { // Insertar izquierda  
        raiz.izquierdo = insertar(raiz.izquierdo, valor);  
    } else { // Insertar derecha  
        raiz.derecho = insertar(raiz.derecho, valor);  
    }  
    return raiz;  
}  
}
```

La función *insertar()* siguió exactamente las mismas reglas del BST utilizadas en C++.

La diferencia principal fue que Java administra automáticamente la memoria mediante el Garbage Collector.

EJERCICIO 3:

Cálculo de altura del árbol:

En Java se utilizó el método *Math.max()* para comparar ambas alturas:

```
return 1 + Math.max(alturaIzquierda, alturaDerecha);
```

Función para calcular altura:

```
public static int calcularAltura(Nodo raiz) {  
  
    if (raiz == null) { // Caso base  
        return 0;  
    }  
    int izquierda = calcularAltura(raiz.izquierdo); // Altura izquierda  
    int derecha = calcularAltura(raiz.derecho); // Altura derecha  
    return 1 + Math.max(izquierda, derecha); // Retornar altura mayor  
}  
}
```

La función recorrió todos los nodos del árbol hasta calcular la profundidad máxima. La implementación fue similar a C++, aunque utilizando referencias en lugar de punteros.

EJERCICIO 4:

Recorrido In-Order:

En Java se utilizó una lista dinámica *List<Integer>*.
List<Integer> resultado = new ArrayList<>();

La función recursiva realizó:

1. Recorrido izquierdo.
2. Guardado del nodo actual.
3. Recorrido derecho.



Función para realizar el recorrido in-order:

```
public static void inOrderAux(Nodo nodo, List<Integer> resultado) { // Función auxiliar

    if (nodo == null) { // Caso base
        return;
    }
    inOrderAux(nodo.izquierdo, resultado); // Izquierda
    resultado.add(nodo.valor); // Nodo actual
    inOrderAux(nodo.derecho, resultado); // Derecha
}

public static List<Integer> recorridoInOrder(Nodo raiz) { // Función principal
    List<Integer> resultado = new ArrayList<>();
    inOrderAux(raiz, resultado);
    return resultado;
}
}
```

Los resultados fueron almacenados en la lista y posteriormente impresos en consola.

EJERCICIO 5:

Inversión de árbol binario:

En Java se utilizó la misma lógica de intercambio, pero utilizando referencias a objetos.

```
Nodo temporal = raiz.izquierdo;
raiz.izquierdo = raiz.derecho;
raiz.derecho = temporal;
```

Función para invertir un árbol binario:

```
public static Nodo invertir(Nodo raiz) {

    if (raiz == null) { // Caso base
        return null;
    }
    Nodo temporal = raiz.izquierdo; // Intercambio
    raiz.izquierdo = raiz.derecho;
    raiz.derecho = temporal;

    invertir(raiz.izquierdo); // Recursividad
    invertir(raiz.derecho);
    return raiz;
}
}
```

La función recursiva permitió convertir el árbol original en un árbol espejo recorriendo automáticamente todos los nodos.

6. CAPTURAS EN C++:

Se incluyen capturas del funcionamiento de cada programa (ejercicios):



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



a) Ejercicio 1 En C++:

```
=====
ARBOL N-ARIO
=====
|-- 1
|  |--2
|  |  |--5
|  |  |-- 6
|  |--3
|  |-- 4
=====
RESULTADO DE LA PRUEBA
=====
Nodos esperados : 6
Nodos calculados: 6
Resultado: CORRECTO
=====
Process exited after 0.03142 seconds with return value 0
Presione una tecla para continuar . . . |
```

b) Ejercicio 2 En C++:

```
=====
ARBOL BINARIO BST
=====
|-- 10
|  |-- 5
|  |  |-- 3
|  |-- 15
=====
VALIDACION BST
=====
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izquierdo del 5 (Esperado 3): 3
=====
Process exited after 0.03173 seconds with return value 0
Presione una tecla para continuar . . . |
```

c) Ejercicio 3 En C++:

```
=====
ARBOL BINARIO
=====
\-- 1
   \-- 2
      \-- 3
=====
RESULTADO DE ALTURA
=====
(CASO 1) Altura esperada : 3
(CASO 1) Altura calculada: 3
(CASO 2) Altura de arbol nulo (esperado 0): 0
Resultado: CORRECTO
=====
Process exited after 0.02698 seconds with return value 0
Presione una tecla para continuar . . . |
```

d) Ejercicio 4 En C++:

```
=====
ARBOL BINARIO
=====
\-- 4
   |-- 2
   |  |-- 1
   |  \-- 3
   \-- 6
      |-- 5
      \-- 7
=====
RECORRIDO IN-ORDER
=====
Resultado esperado: 1 2 3 4 5 6 7
Resultado obtenido: 1 2 3 4 5 6 7
Resultado: CORRECTO
=====
Process exited after 0.03053 seconds with return value 0
Presione una tecla para continuar . . . |
```




e) Ejercicio 5 En C++:

```
=====
ARBOL ORIGINAL
=====
\-- 1
   |-- 2
   \-- 3

Antes de invertir:
Hijo Izq: 2 | Hijo Der: 3

=====
ARBOL INVERTIDO
=====
\-- 1
   |-- 3
   \-- 2

Despues de invertir (Esperado: Izq 3 | Der 2)
Hijo Izq: 3 | Hijo Der: 2
Resultado: CORRECTO

=====
Process exited after 0.05632 seconds with return value 0
Presione una tecla para continuar . . . |
```

7. CAPTURAS EN JAVA:

Se incluyen capturas del funcionamiento de cada programa (ejercicios):

a) Ejercicio 1 En Java:

```
=====
ARBOL N-ARIO
=====
\-- 1
   |-- 2
   |  |-- 5
   |  |-- 6
   |-- 3
   \-- 4

=====
RESULTADO
=====
Nodos esperados : 6
Nodos calculados: 6
Resultado: CORRECTO
```

b) Ejercicio 2 En Java:

```
=====
ARBOL BST
=====
\-- 10
   |-- 5
   |  |-- 3
   |  |-- 15

=====
RESULTADO
=====
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izquierdo del 5 (Esperado 3): 3
Resultado: CORRECTO
```

c) Ejercicio 3 En Java:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
=====
ARBOL BINARIO
=====
\-- 1
  \-- 2
    \-- 3

=====
RESULTADO
=====
(CASO 1) Altura esperada : 3
(CASO 1) Altura calculada: 3
(CASO 2) Altura de arbol nulo (esperado 0): 0
Resultado: CORRECTO
```

d) Ejercicio 4 En Java:

```
=====
ARBOL BINARIO
=====
\-- 4
  |-- 2
  |   |-- 1
  |   \-- 3
  \-- 6
      |-- 5
      \-- 7

=====
RECORRIDO IN-ORDER
=====
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]
Resultado obtenido: [1, 2, 3, 4, 5, 6, 7]
Resultado: CORRECTO
```

e) Ejercicio 5 En Java:

```
=====
ARBOL ORIGINAL
=====
\-- 1
  |-- 2
  \-- 3

Antes de invertir:
Hijo Izq: 2 | Hijo Der: 3

=====
ARBOL INVERTIDO
=====
\-- 1
  |-- 3
  \-- 2

Después de invertir (Esperado: Izq 3 | Der 2)
Hijo Izq: 3 | Hijo Der: 2
Resultado: CORRECTO
```

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones



- La práctica realizada permitió comprender de manera clara el funcionamiento de las estructuras de datos tipo árbol, identificando cómo se organizan jerárquicamente los nodos y cómo esta organización facilita la representación y manipulación eficiente de información.
- El desarrollo de los ejercicios fortaleció el uso de la recursividad como herramienta fundamental para trabajar con estructuras jerárquicas. A través de operaciones como conteo de nodos, cálculo de altura, recorridos e inversión de árboles, se evidenció que la recursividad simplifica la lógica de programación y permite recorrer estructuras complejas de forma más ordenada, eficiente y comprensible.
- La implementación de los ejercicios en C++ y Java permitió comparar directamente ambos lenguajes de programación, identificando diferencias en aspectos como el manejo de memoria, uso de punteros y referencias, administración de objetos y sintaxis. Esto ayudó a comprender que, aunque la lógica algorítmica puede mantenerse igual, cada lenguaje posee características que influyen en la forma de implementar estructuras dinámicas como los árboles.
- El trabajo realizado sobre Árboles Binarios de Búsqueda (BST) permitió entender la importancia de mantener una correcta organización de los nodos para optimizar operaciones de búsqueda, inserción y recorrido. Asimismo, el recorrido In-Order demostró cómo los árboles pueden utilizarse para almacenar y recuperar información de manera ordenada.
- Finalmente, la práctica contribuyó al fortalecimiento de habilidades de análisis lógico, resolución de problemas y programación estructurada, ya que cada ejercicio requirió comprender el comportamiento interno del árbol, analizar casos base y aplicar correctamente funciones recursivas.

2.10 Recomendaciones

- Continuar practicando el desarrollo de estructuras de datos tipo árbol mediante ejercicios más complejos, con el fin de fortalecer la comprensión de la recursividad, el manejo de nodos y la lógica necesaria para resolver problemas relacionados con estructuras jerárquicas y dinámicas.
- Implementar otros tipos de recorridos como Pre-Order y Post-Order para ampliar el conocimiento sobre las diferentes formas de acceder y procesar la información almacenada dentro de un árbol binario, identificando las ventajas y aplicaciones de cada recorrido.
- Fortalecer el manejo de memoria dinámica en C++, especialmente en el uso correcto de punteros y liberación de memoria, para evitar errores comunes como fugas de memoria, referencias inválidas o accesos incorrectos a nodos.
- Continuar desarrollando ejercicios que involucren recursividad, debido a que esta técnica es fundamental en estructuras jerárquicas y constituye una de las bases más importantes para resolver problemas avanzados de programación y algoritmos.

2.11 Referencias bibliográficas

- [1] Enjoy Algorithms, “Binary Tree Traversals: Preorder, Inorder and Postorder,” 2023. [En línea]. Disponible en: <https://www.enjoyalgorithms.com/blog/binary-tree-traversals-preorder-inorder-postorder> [Accedido: 07-may-2026].
- [2] EnjoyAlgorithms, “Find Height of a Binary Tree”, 2023. [En línea]. Disponible en: <https://www.enjoyalgorithms.com/blog/find-height-of-a-binary-tree> [Accedido: 13-may-2026].
- [3] GeeksforGeeks, “Inorder Traversal of Binary Tree,” 2025. [En línea]. Disponible en: <https://www.geeksforgeeks.org/dsa/inorder-traversal-of-binary-tree/> [Accedido: 07-may-2026].
- [4] Interview Cake, “Binary Tree,” 2024. [En línea]. Disponible en: <https://www.interviewcake.com/concept/java/binary-tree> [Accedido: 07-may-2026].



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



- [5] TechVidvan, “Tree in Data Structure,” s.f. [En línea]. Disponible en: <https://techvidvan.com/tutorials/tree-in-data-structure/> [Accedido: 13-may-2026].
- [6] Wikipedia, “Árbol binario,” 2025. [En línea]. Disponible en: https://es.wikipedia.org/wiki/%C3%81rbol_binario [Accedido: 07-may-2026].
- [7] Wikipedia, “Árbol (informática)”, 2024. [En línea]. Disponible en: [https://es.wikipedia.org/wiki/%C3%81rbol_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/%C3%81rbol_(inform%C3%A1tica)) [Accedido: 13-may-2026].

2.12 Anexos

GitHub Personal:

